CSE 574A: Intro to Machine Learning
Project 5

Competitive Learning Based Image Compressor

Submitted By: Viral Koshti
Student ID: 50761354

1 Method

The provided `barbara.png` was read as an 8-bit grayscale image, giving an image matrix $A \in \{0, \ldots, 255\}^{512\times512}$. The image was not normalized or intensity scaled. It was split into non-overlapping $d \times d$ blocks. With $d = 4$, this produces

$$N = \left(\frac{512}{4}\right)^2 = 16{,}384$$

training blocks, each containing $d^2 = 16$ pixel intensities.

Each of the K neurons stores a 4×4 reference block. In the code, this is represented as a vector of 16 weights. For input block A_{ij}, the winning neuron is the one with the smallest squared Euclidean distance:

$$k^*(i,j) = \arg\min_{1\le k\le K} \sum_{u=1}^{d}\sum_{v=1}^{d} \left(A_{ij}[u,v] - X_k[u,v]\right)^2 . \tag{1}$$

Only the winning neuron is updated. Its weights move toward the input block using the required learning rate $\eta = 0.5$:

$$X_{k^*} \leftarrow (1-\eta)X_{k^*} + \eta A_{ij} = 0.5X_{k^*} + 0.5A_{ij}. \tag{2}$$

The blocks are presented in a random order for 35 epochs. A neuron that receives no winning assignments during an epoch is reinitialized using a training block with high reconstruction error. The random seed is set to 574 so that the experiments can be reproduced.

After each epoch, each original block is replaced by its nearest learned reference block. The weights are not updated during this step. The reconstructed image is called A', and its root mean squared pixel error is

$$\text{RMS} = \sqrt{\frac{1}{512^2}\sum_{r=1}^{512}\sum_{c=1}^{512}(A'_{rc} - A_{rc})^2} = \frac{\|A' - A\|_F}{512}, \tag{3}$$

where $\|\cdot\|_F$ denotes the Frobenius norm.

2 Q1: Bits Needed for the Original Image

The original image contains $512 \times 512 = 262{,}144$ pixels. Since each pixel can take any of 256 intensity values from 0 to 255, it needs $\log_2 256 = 8$ bits. Therefore,

$$B_{\text{original}} = 512 \times 512 \times 8 = \boxed{2{,}097{,}152 \text{ bits}}. \tag{4}$$

This equals 262,144 bytes, or 256 KiB. It is the size of the raw pixel array, not the size of the PNG file, which may include metadata and use lossless compression.

3 Q2: Bits Needed for the Compressed Representation

For $d = 4$, the image contains $N = 16{,}384$ blocks. Instead of storing the 16 original pixel values for every block, the compressed representation stores an index identifying one of the K learned reference blocks. The reference blocks themselves are stored once in a shared codebook.

A fixed-length index requires $\lceil \log_2 K \rceil$ bits. For all 16,384 blocks, the indices therefore require $16{,}384\lceil \log_2 K \rceil$ bits. If each codebook pixel is quantized and stored with 8 bits, one 4×4 reference block costs $16 \times 8 = 128$ bits, making the total codebook cost $128K$ bits. Thus,

$$\boxed{B_{\text{compressed}} = 16{,}384\lceil \log_2 K \rceil + 128K \text{ bits.}} \tag{5}$$

Table 1: Storage requirements with 8-bit codebook entries ($d = 4$).

K	Index bits	Codebook bits	Total bits	Ratio
2	16,384	256	16,640	126.03:1
8	49,152	1,024	50,176	41.80:1
32	81,920	4,096	86,016	24.38:1

The training code keeps the weights as floating-point values, and these are the values used to calculate the reported RMS. The 8-bit codebook assumption is for the storage calculation only. The reported RMS values were not recomputed after quantizing the codebook to 8 bits. If the weights are instead stored as 64-bit floating-point values, the codebook cost is $K \times 16 \times 64 = 1024K$ bits. The bit counts exclude container headers and any other file-format overhead.

4 Convergence of the Modified Competitive Learning Rule

The nearest-neuron rule does *not* guarantee that weights converge to fixed values when the learning rate remains constant at $\eta = 0.5$. This can be shown with a counterexample using only one neuron.

Let $K = 1$, so that the only neuron wins every time. Consider the repeated scalar inputs $0, 1, 0, 1, \ldots$, and let w_t be the weight immediately before the t-th presentation of 0. Following the update rule,

$$w'_t = 0.5w_t + 0.5(0) = \frac{w_t}{2}, \tag{6}$$

$$w_{t+1} = 0.5w'_t + 0.5(1) = \frac{w_t}{4} + \frac{1}{2}. \tag{7}$$

The values sampled once per two-input cycle therefore satisfy

$$w_{t+1} - \frac{2}{3} = \frac{1}{4}\left(w_t - \frac{2}{3}\right), \tag{8}$$

which gives $w_t \to 2/3$. Yet the intermediate values satisfy $w'_t = w_t/2 \to 1/3$. Since the weights approach two different values on alternating updates, the entire sequence does not converge to a single fixed value. This example obeys the distance-based winner selection and the specified weight update, so convergence is not guaranteed in general.

For comparison, when the same input x is always presented to a winning neuron, the recurrence gives

$$w_t = x + (1 - \eta)^t(w_0 - x). \tag{9}$$

Because $0 < \eta < 1$, the second term tends to zero and $w_t \to x$. So the weights can converge when the input stays the same, but convergence is not guaranteed when the inputs vary and the learning rate stays fixed.

5 Experimental Results

Experiments used $d = 4$, $\eta = 0.5$, 35 epochs, and $K \in \{2, 8, 32\}$. The final reconstructed images are shown in Figure 1. Figure 2 plots the RMS recorded after every epoch.

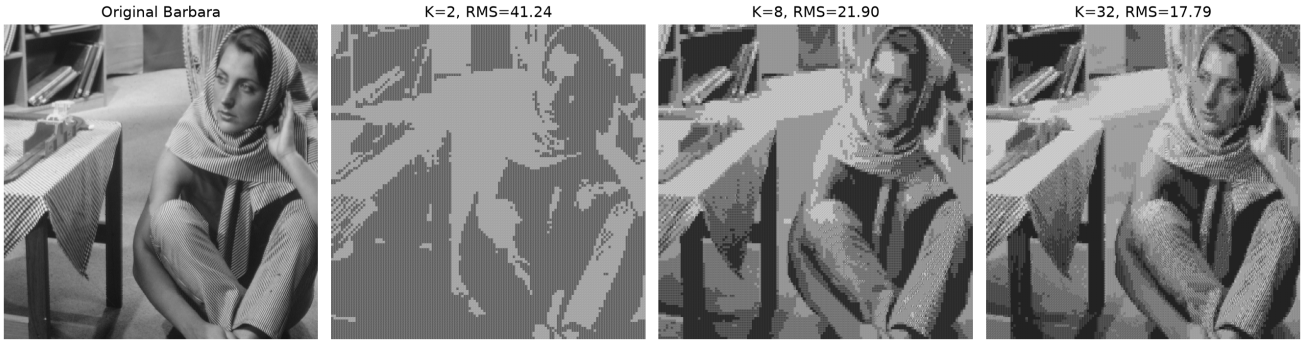


Figure 1: Original Barbara image and reconstructed images for $d = 4$ with $K = 2$, $K = 8$, and $K = 32$.

Table 2: Measured reconstruction error after 35 epochs.

K	Final RMS	Lowest observed RMS	Epoch of lowest RMS
2	41.240394	34.146854	10
8	21.902699	20.744394	15
32	17.788799	17.453383	15

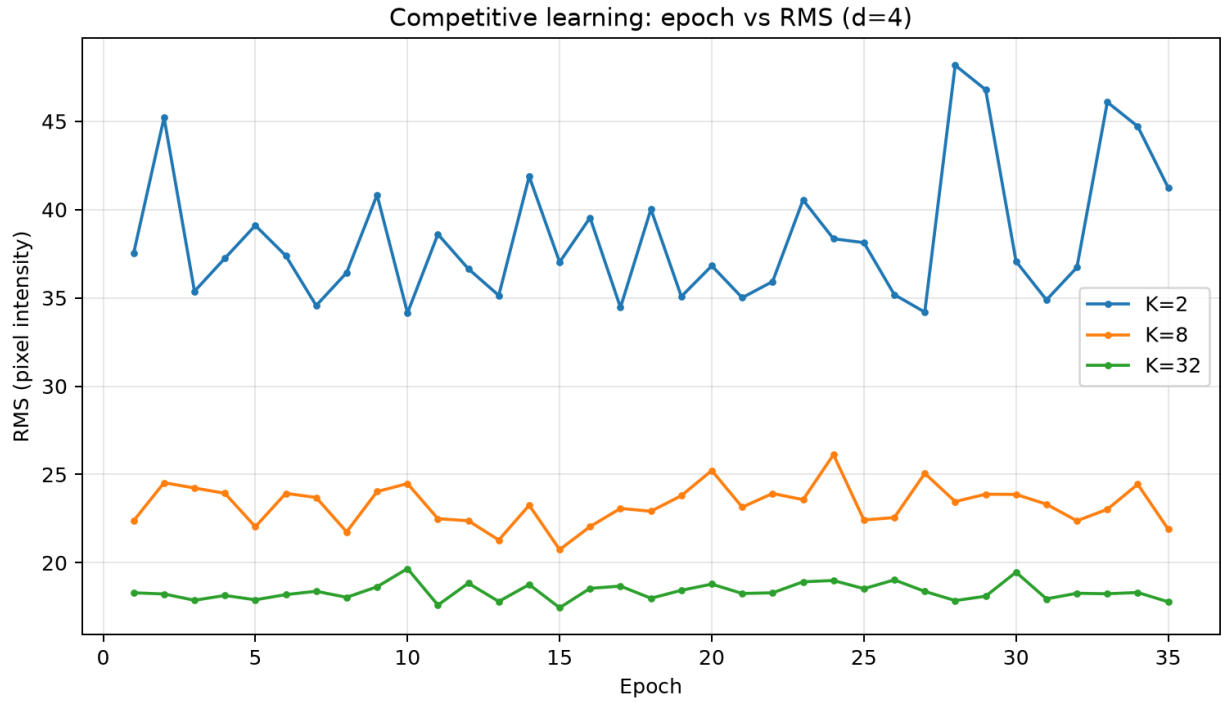


Figure 2: Epoch number versus RMS for the three codebook sizes.

The $K = 2$ image loses a lot of detail and mostly captures changes in overall intensity. With $K = 8$, the shapes become clearer and some textured regions are easier to recognize. At $K = 32$, more fabric patterns and edges are preserved, and the final RMS is lower. This improvement comes at a cost: a larger codebook needs more bits to store its reference blocks and the block indices.

The RMS does not decrease in every epoch. Because training updates only the winner for each block and uses a fixed learning rate, an update that helps one block can make the reconstruction of other blocks worse. As a result, the last epoch is not necessarily the one with the lowest RMS.

6 Q3: How Competitive Learning Compresses the Image

Nearby parts of an image often contain similar patterns, such as smooth regions, edges, and repeated textures. Competitive learning uses these similarities to represent many 4×4 image patches with a smaller set of learned prototype patches.

When a patch is presented, the closest neuron wins, and its weights move toward that patch. After training, the learned neurons serve as representatives for the image patches. Instead of storing 16 separate 8-bit pixel values for each patch, the encoder stores a short index. During decoding, this index is used to retrieve the corresponding 4×4 prototype and build the reconstructed image.

This compression is lossy. Different patches may share the same prototype, changing some pixel values and smoothing out fine textures and edges. Increasing K gives the model more prototypes to choose from, which can improve image quality but reduces the compression ratio.

7 Reproducibility

The implementation is in `Proj5.py`. It uses Python 3, NumPy, Pillow, Matplotlib, and Numba. Place `barbara.png` in the same folder as the script. The default configuration is $d = 4$, $K = 8$, 35 epochs, $\eta = 0.5$, and seed 574. The script prints one RMS value after each epoch and displays the final reconstruction. Changing the `NEURONS` setting to 2 or 32 reproduces the other experiment configurations.